

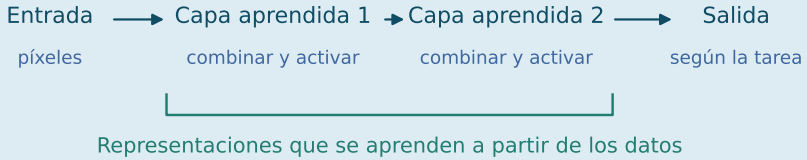
Aprendizaje Automático

Deep Learning, autoencoders y representaciones

22 de octubre de 2026

Eloy Alvarado Narváez

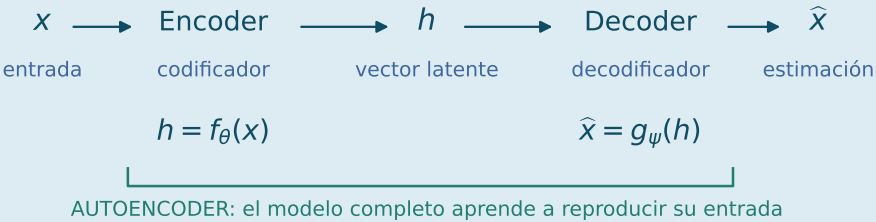
De redes neuronales a deep learning



- Deep learning: aprendizaje con redes que componen varias capas de transformación.
- Las representaciones intermedias se ajustan mediante los datos y el objetivo.

Mayor profundidad permite más composiciones; su utilidad debe evaluarse.

Qué es un autoencoder

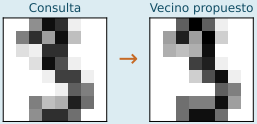


- Encoder (codificador): calcula una representación de la entrada.
- Decoder (decodificador): usa esa representación para estimar la entrada.

Encoder: función. Vector latente: resultado. Programa: implementación.

Para qué nos puede servir un autoencoder

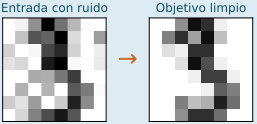
Buscar imágenes parecidas



Comparar vectores latentes

El parecido útil depende de la tarea.

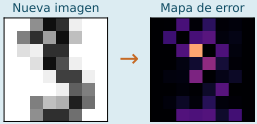
Recuperar una señal limpia



Entrenar con pares ruido / limpio

Es otra tarea de entrenamiento.

Priorizar una revisión



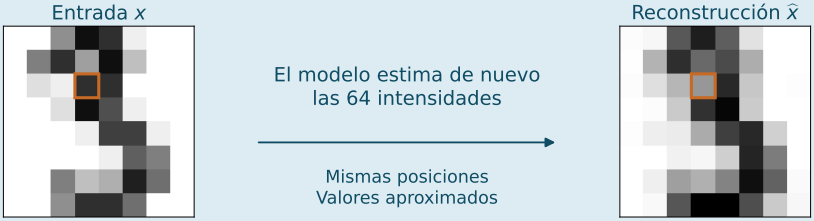
Error alto → revisar la causa

Una alerta no confirma un defecto.

Esquemas de uso: aprender a reconstruir es un medio para obtener una herramienta útil.

El uso decide qué debemos conservar y cómo comprobar si el modelo ayuda.

Qué significa reconstruir una imagen

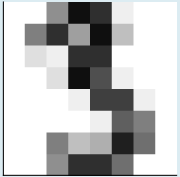


Cuadro naranja: el mismo píxel (fila 3, columna 4). 0 = blanco; 1 = negro.

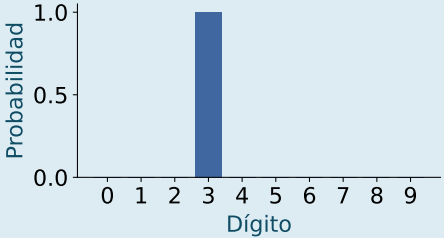
Reconstruir: estimar cada valor de la entrada y volver a colocarlo en su posición.

Clasificar y reconstruir

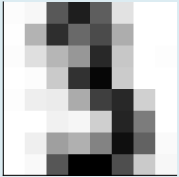
Imagen de entrada



Clasificación: 10 salidas



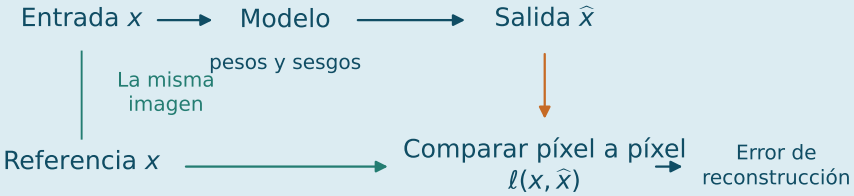
Reconstrucción: 64 píxeles



La misma imagen de la clase de clasificación. Clasificar produce diez probabilidades; reconstruir produce 64 intensidades.

La etiqueta del dígito no entra en la pérdida de reconstrucción.

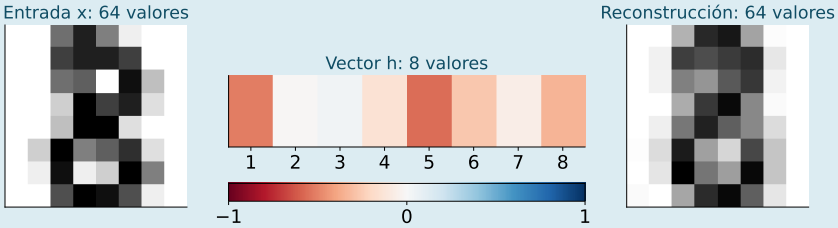
La entrada también es la referencia



- x entra al modelo y se conserva para evaluar su salida $\widehat{x}$.
- La etiqueta del dígito no interviene en $\ell(x, \widehat{x})$.

La referencia se conoce; la salida debe aprenderse.

Encoder, vector latente y decoder

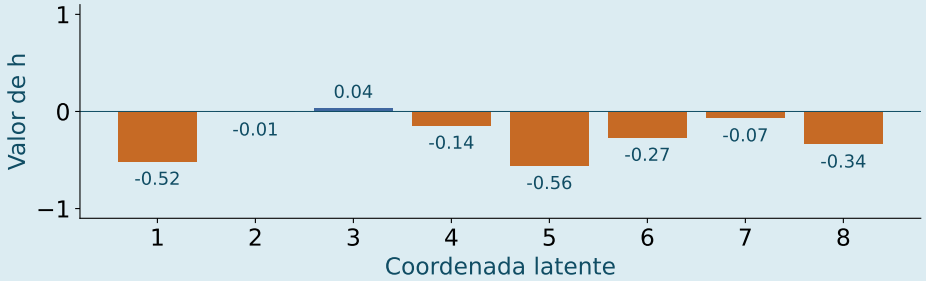


$h = f_\theta(x)$: codificar. $\widehat{x} = g_\psi(h)$: decodificar.

- Elegimos 8 unidades en el vector latente: cada imagen produce 8 activaciones.

El decoder recibe solo el vector latente. Ambas partes aprenden con el error final.

Leer las 8 coordenadas latentes



Vector latente calculado para el mismo ocho de la lámina anterior. Las coordenadas pueden ser negativas.

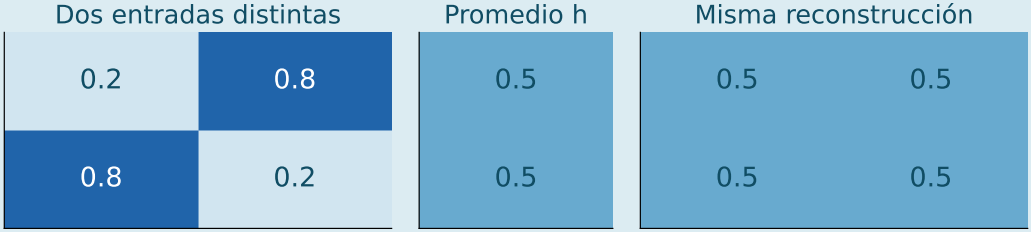
Son activaciones calculadas con la entrada y los pesos; no son ocho probabilidades.

Dimensión latente y capacidad

Restricción	Qué controla
Vector latente pequeño	Cantidad de coordenadas: aquí elegimos 8.
Ancho y profundidad	Flexibilidad del encoder y decoder.
Regularización	Preferencias adicionales sobre el ajuste.

Ocho coordenadas reales no equivalen a ocho bits.

Un resumen puede perder información

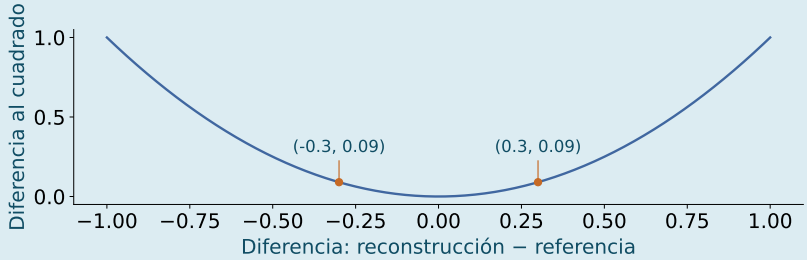


Encoder fijo: $h = (x_1 + x_2)/2$. Decoder fijo: $\widehat{x} = (h, h)$.

Al coincidir los vectores latentes, el decoder determinista entrega la misma salida.

Error cuadrático de reconstrucción

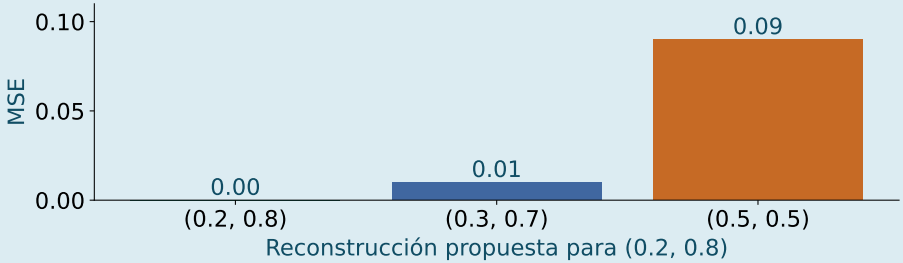
$$\ell_i = \frac{1}{64}\sum_{j=1}^{64}(\widehat{x}_{ij} - x_{ij})^2, \qquad L = \frac{1}{n}\sum_i \ell_i.$$



Cada diferencia se eleva al cuadrado antes de promediar. Los signos no se cancelan.

La pérdida del ejemplo pequeño

$$\ell = \frac{(0.5 - 0.2)^2 + (0.5 - 0.8)^2}{2} = \frac{0.09 + 0.09}{2} = 0.09.$$



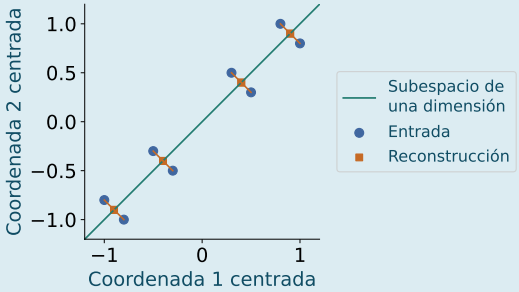
La misma referencia (0.2, 0.8), tres reconstrucciones propuestas.

Una copia perfecta puede aportar poco

- Una identidad sin restricciones devuelve la entrada con error cero.
- Si conserva todos los detalles, puede no resumir nada útil.
- Un vector latente pequeño también puede memorizar con suficiente flexibilidad.

Necesitamos una restricción adecuada y una evaluación fuera del ajuste.

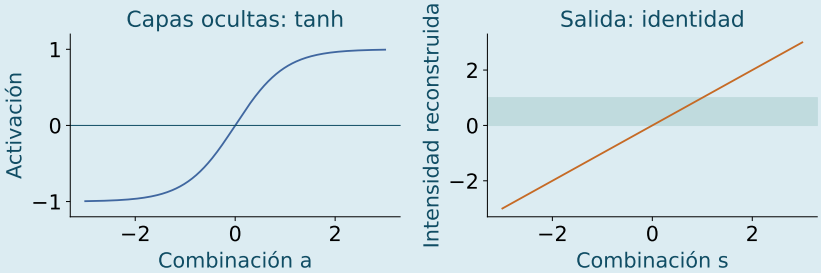
PCA como referencia de reconstrucción



Datos artificiales centrados. PCA proyecta sobre una dirección y reconstruye en ese subespacio.

$$z = (x - \overline{x})P_k, \qquad \widehat{x} = \overline{x} + zP_k^T.$$

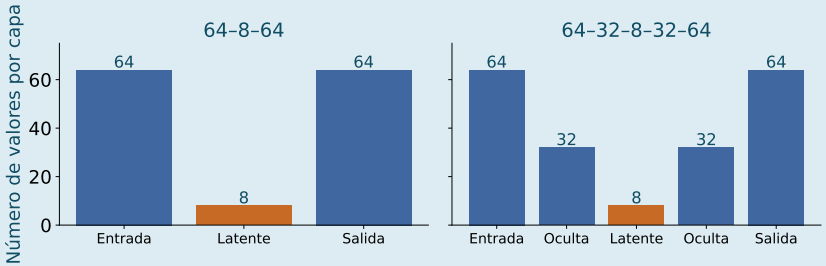
Una representación no lineal



La tangente hiperbólica acota las activaciones ocultas. La salida lineal puede exceder [0,1].

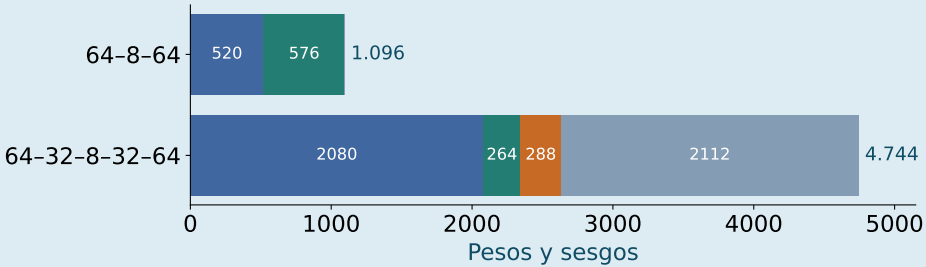
Conservamos las salidas completas al calcular el MSE.

Qué aporta la profundidad



- 8 unidades: tamaño del vector latente elegido en ambas redes.
- 32 unidades: ancho elegido para las capas adicionales; sus pesos son independientes.

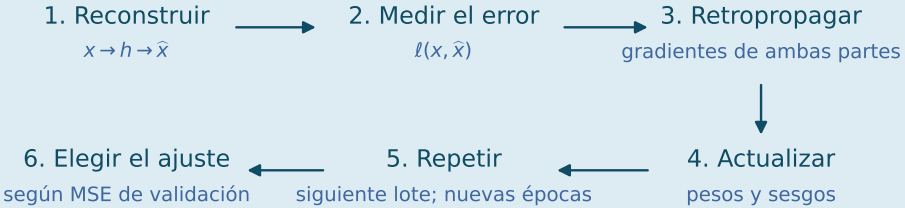
Cuántos parámetros aprende cada arquitectura



- Ambas arquitecturas resumen cada imagen con 8 valores.
- La red con capas adicionales aprende más pesos y sesgos.

El tamaño del vector latente y la cantidad de parámetros son cosas distintas.

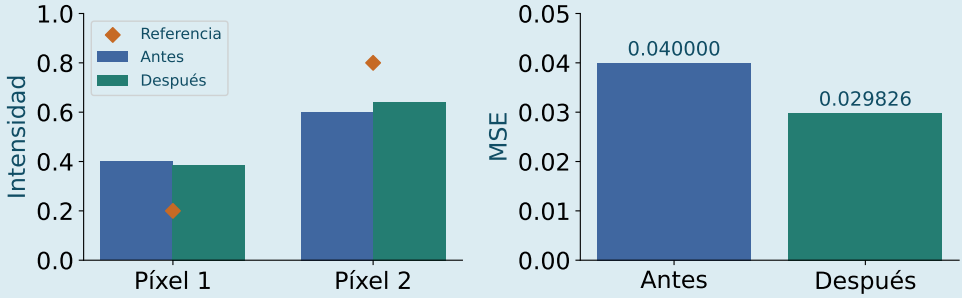
Cómo se entrena un autoencoder



- Inicializar los parámetros una vez; continuar desde ellos en cada actualización.
- Al terminar cada época, medir validación sin actualizar los parámetros.

La pérdida de reconstrucción guía a las dos partes del modelo.

Qué cambia después de aprender del error



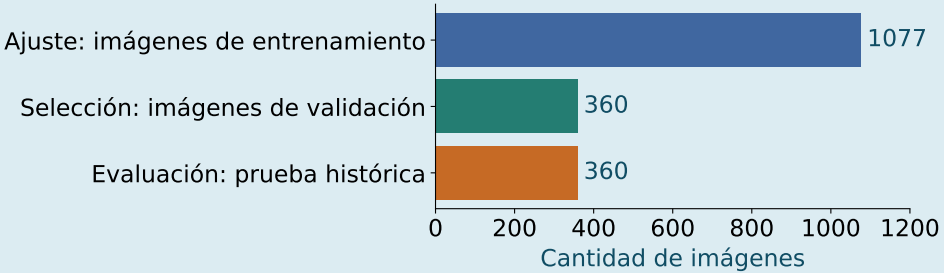
En este paso, las salidas se acercan a sus referencias y el MSE baja de 0.0400 a 0.0298.

Sin etiquetas también puede haber fuga

- Ajustar PCA con todas las imágenes incorpora la geometría de prueba.
- Entrenar el autoencoder con la reserva cambia qué ejemplos ha visto.
- Elegir la dimensión mirando resultados de prueba también usa esa reserva.

El protocolo depende de qué información estará disponible en el uso previsto.

El protocolo de esta sesión



PCA y redes se ajustan con entrenamiento. Validación elige arquitectura y época por MSE.

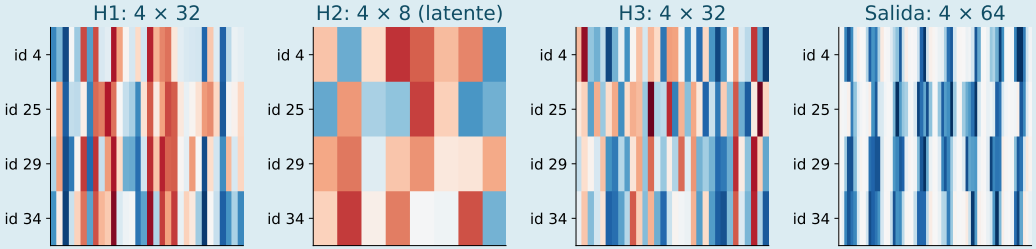
Las etiquetas aparecen después, en una evaluación de utilidad con el encoder fijo.

Receta del ejemplo numérico: Python y Adam

Elemento	Elección
Redes	$64 \to 8 \to 64$ y $64 \to 32 \to 8 \to 32 \to 64$.
Activaciones	Tangente hiperbólica oculta y salida lineal.
Entrenamiento	Adam; lotes de 64 imágenes; tasa 0.002.
Regularización	Penalización de pesos con $\alpha = 0.0001$.
Presupuesto	Hasta 150 épocas por candidata.
Referencias	Imagen media y PCA con 8 componentes.

R reproduce la tarea con RSNNS y descenso por gradiente; sus resultados pueden diferir.

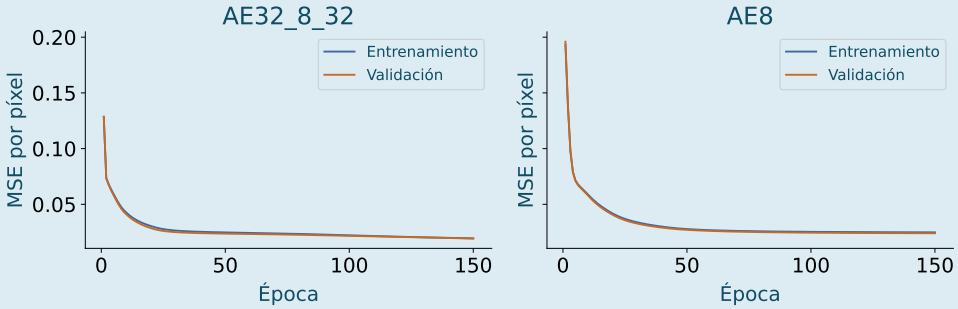
De la imagen a su reconstrucción



- Cada fila sigue a la misma imagen a través de la red.
- La segunda matriz contiene sus 8 coordenadas latentes.

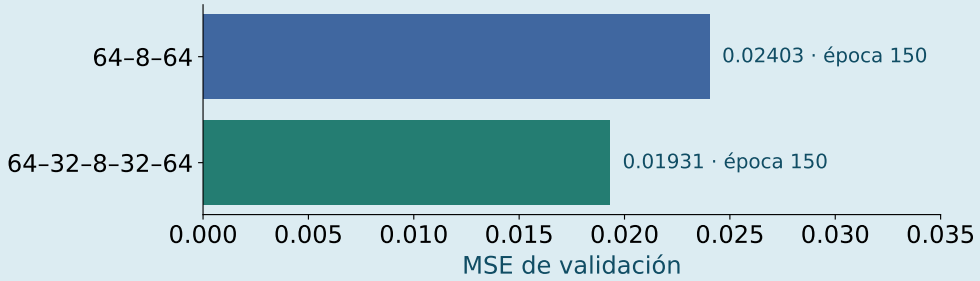
Los pesos están fijos; las activaciones cambian con cada imagen.

Cómo evolucionó la reconstrucción · Python



El error reportado es MSE por píxel, sin penalización.

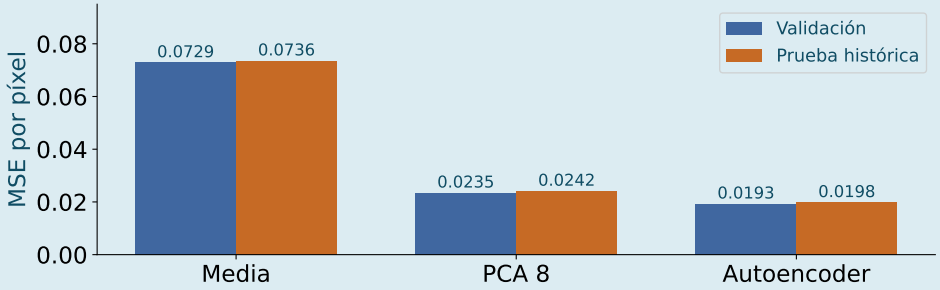
La selección llega al límite de épocas



Cada barra muestra el menor MSE de validación observado dentro de 150 épocas.

Ambos mínimos están en la última época: el presupuesto limita la conclusión.

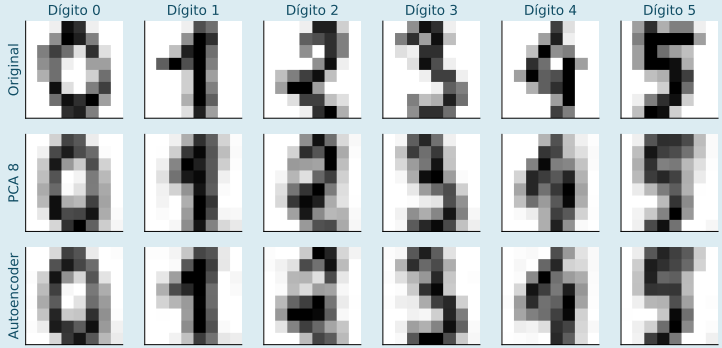
Reconstrucción frente a referencias · Python



Referencias y red sobre las mismas imágenes y la misma escala. Menor MSE es mejor.

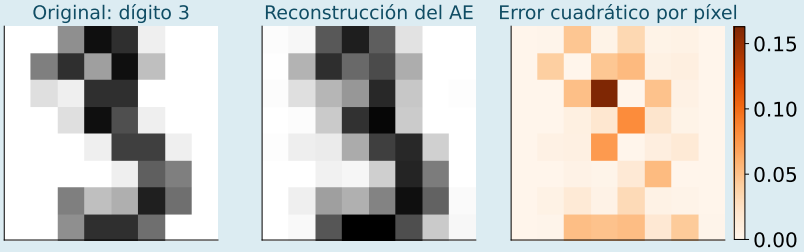
Reconstrucción es la primera pregunta; aún falta evaluar la utilidad del vector latente.

Qué se pierde en la reconstrucción



Primer caso de cada dígito 0 a 5 en prueba. Solo la visualización recorta intensidades a [0,1]; las métricas no.

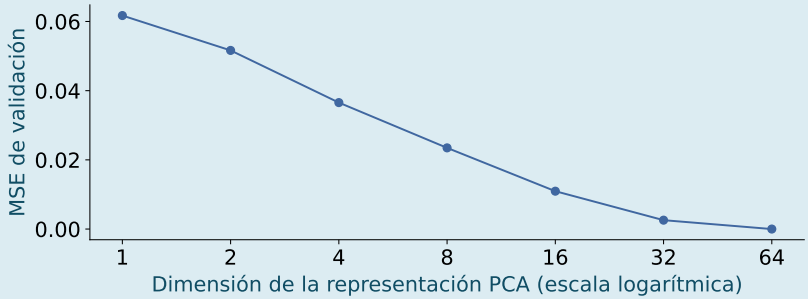
Dónde se acumula el error de una imagen



Píxel señalado al inicio: $(0.4089 - 0.8125)^2 \approx 0.1629$.

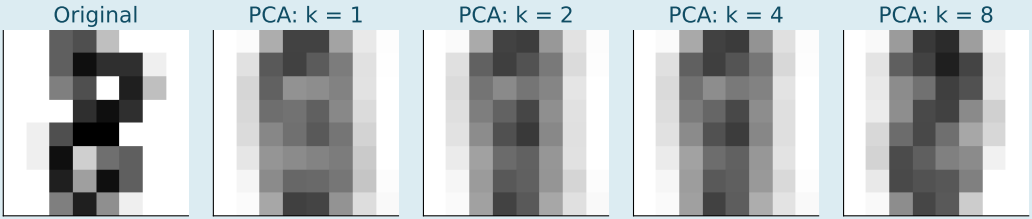
MSE de la imagen: 0.017220. Promediamos las 64 contribuciones del mapa.

Dimensión y error de reconstrucción



PCA ajustado en entrenamiento. La curva usa solamente validación; k=8 se fijó para el experimento principal.

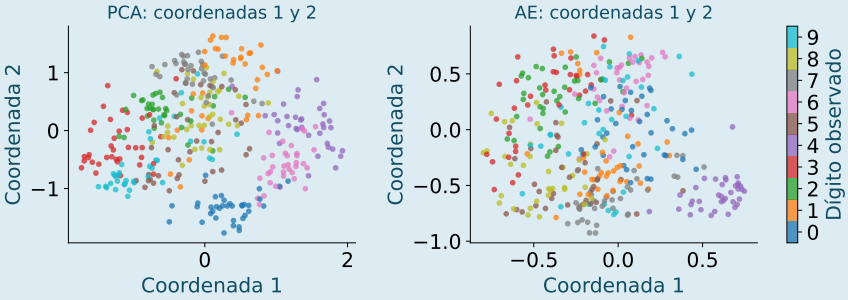
La dimensión también se puede ver en una imagen



Un mismo ocho de prueba con 1, 2, 4 y 8 direcciones de PCA. Todas se obtienen de las direcciones guardadas.

Más detalle visible no determina por sí solo la dimensión conveniente para cada uso.

Mirar 2 coordenadas latentes



Se muestran 2 de 8 coordenadas. Los colores son etiquetas añadidas para inspección, no usadas por el autoencoder.

Una representación puede servir para otra tarea

Uso del vector latente	Evaluación apropiada
Reconstruir	Error sobre la entrada original.
Clasificar	Métricas de clasificación con etiquetas.
Buscar vecinos	Relevancia de los objetos recuperados.
Generar alertas	Errores y utilidad en condiciones reales.

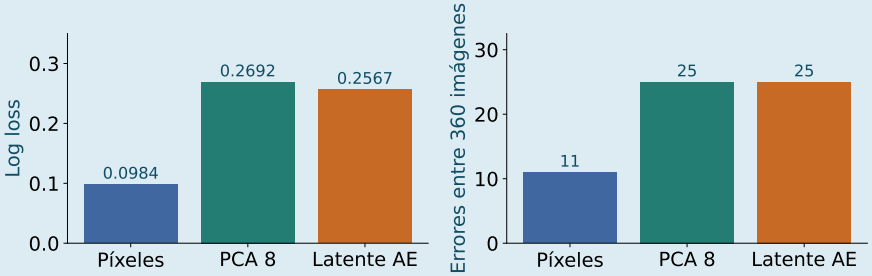
La utilidad depende de la tarea posterior.

Clasificar con representaciones fijas

- Congelar PCA y el autoencoder ya ajustados.
- Transformar entrenamiento, validación y prueba con esas funciones.
- Ajustar un escalado y una logística con C=1 en entrenamiento.
- Comparar píxeles, PCA8 y vector latente AE usando la misma receta supervisada.

Las etiquetas entrenan la logística, sin volver a ajustar el encoder.

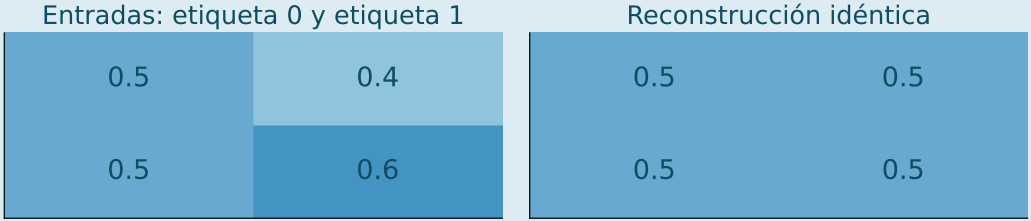
La utilidad para clasificar dígitos · Python



Logística con estandarización en entrenamiento y C=1. Prueba histórica de 360 imágenes.

Los dos vectores latentes producen 25 errores; los píxeles, 11. La utilidad depende de la receta.

Reconstrucción y discriminación

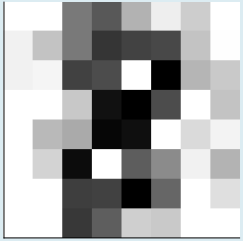


Ejemplo artificial: etiqueta $y = 1$ si $x_2 > 0.5$. Conservar solo $h = x_1$ pierde esa distinción.

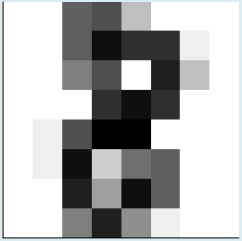
Ambas reconstrucciones tienen MSE 0.005, pero el vector latente es el mismo para las dos etiquetas.

Reconstrucción a partir de una entrada alterada

Entrada alterada (ejemplo)



Referencia limpia del objetivo



Variante conceptual: la entrada se altera, pero la referencia de la pérdida permanece limpia.

$$\widetilde{x} \longmapsto g_\psi(f_\theta(\widetilde{x})), \qquad \ell(x, \widehat{x}).$$

La imagen limpia de la derecha es el objetivo, no una salida obtenida por nuestra red.

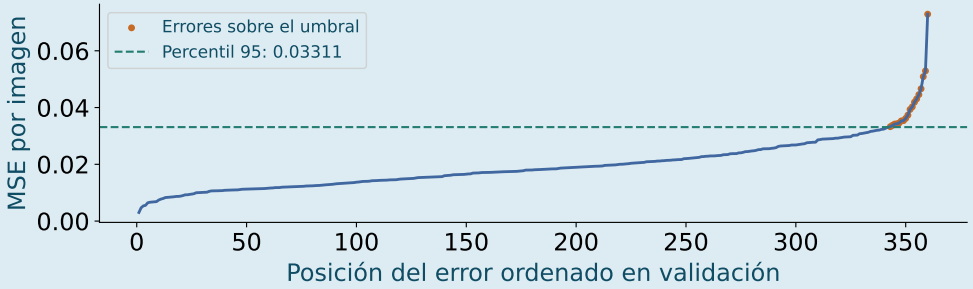
Un error de reconstrucción como señal

$$e(x) = \frac{1}{64} \sum_j (x_j - \widehat{x}_j)^2, \qquad \text{alerta si } e(x) > c.$$

- Para detectar anomalías, el ajuste debe representar lo considerado habitual.
- El umbral requiere datos y criterios compatibles con ese uso.
- Un error alto señala mala reconstrucción; la causa necesita revisión.

El puntaje no es una probabilidad de anomalía.

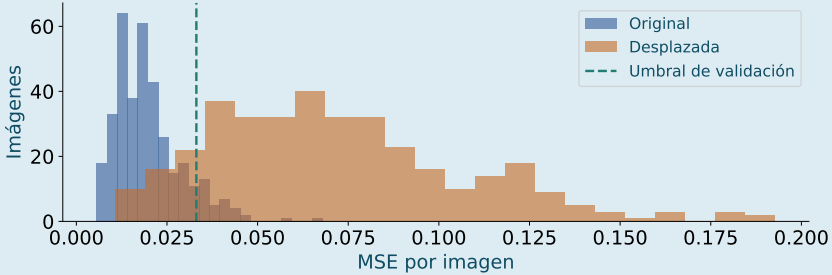
Cómo se fija el umbral de reconstrucción · Python



- Ordenamos los 360 errores de validación de menor a mayor.
- El percentil 95 fija el corte $c \approx 0.033108$.

18 de 360 errores superan el corte: 5 % en esta validación.

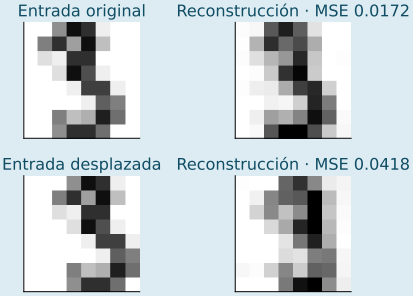
Prueba de estrés y umbral · Python



Umbral: percentil 95 del error de validación, $c = 0.03311$. Desplazamiento artificial de una columna a la derecha.

Alertas: 9.72 % en originales y 88.61 % en desplazadas. No son sensibilidad ni especificidad frente a anomalías reales.

Qué compara el error de una imagen desplazada



Arriba: imagen original y su reconstrucción. Abajo: entrada desplazada y su propia reconstrucción.

En el estrés, el error compara la salida con la entrada desplazada, no con la original limpia.

Cuándo una alerta puede fallar

Situación	Consecuencia posible
Variación habitual no representada	Muchos avisos sobre casos aceptables.
Anomalías presentes en entrenamiento	El modelo puede aprender a reconstruirlas.
Decoder muy flexible	Puede reconstruir casos fuera de lo esperado.
Cambio de adquisición	El puntaje refleja el cambio técnico.

Se necesita una validación específica del sistema de alertas.

Convoluciones y estructura de una imagen



Multiplicar posición a posición y sumar: $1(1) + 0(-1) + 1(1) + 0(-1) = 2$.

- El mismo filtro se aplica en otras posiciones.
- Nuestra red es densa; aquí introducimos la idea de pesos compartidos.

Reconstrucción y generación

- Nuestro decoder recibe vectores latentes producidos por imágenes.
- Un vector elegido al azar puede caer en una región que el modelo no aprendió a usar.
- Los modelos generativos añaden supuestos u objetivos para describir cómo producir muestras.

Un autoencoder determinista no define automáticamente una distribución útil para generar imágenes.

Un sistema hipotético de inspección

- Se quiere detectar daños en envases usando fotografías.
- Solo hay imágenes históricas de productos aceptados por una cámara.
- La propuesta usa un autoencoder y rechaza el 5 % con mayor error.
- Se anuncia detección de cualquier defecto con un 95 % de confianza.

¿Qué demuestra el entrenamiento disponible y qué falta evaluar?

Una evaluación que respalde la decisión

- Definir defectos relevantes y consecuencias del rechazo.
- Separar por lote, fecha o unidad de uso cuando corresponda.
- Calibrar el umbral con datos pertinentes y evaluar en una reserva.
- Examinar errores y prever revisión humana antes de automatizar.

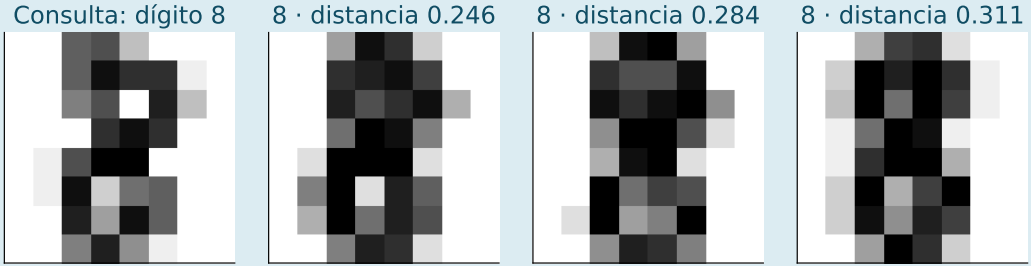
El percentil de un puntaje no es un nivel de confianza sobre el defecto.

Tres preguntas que requieren evidencia distinta · Python

Pregunta	Qué observamos hoy	Qué no se deduce
¿Reconstruye?	MSE del AE menor que PCA8 en esta prueba.	Convergencia o superioridad universal.
¿Sirve para clasificar?	25 errores con ambos vectores latentes; 11 con píxeles.	Utilidad para cualquier clasificador.
¿Detecta anomalías?	Respuesta a un desplazamiento artificial.	Detección validada de defectos reales.

El objetivo de entrenamiento y la evaluación deben corresponder al uso que queremos defender.

Representaciones y similitud



Comparamos los 8 valores del vector latente y ordenamos las imágenes por cercanía.

Programa · leer y separar los datos

Python

```
datos = pd.read_csv(
    CARPETA / "digitos.csv")
X = datos.filter(
    regex="^px").to_numpy() / 16
tr = datos["conjunto"] == "entrenamiento"
va = datos["conjunto"] == "validacion"
X_tr = X[tr]
X_va = X[va]
```

R · RSNNS / stats

```
datos <- read.csv("digitos.csv")
X <- as.matrix(datos[, grepl("^px",
    names(datos))]) / 16
tr <- datos$conjunto == "entrenamiento"
va <- datos$conjunto == "validacion"
X_tr <- X[tr, , drop = FALSE]
X_va <- X[va, , drop = FALSE]
```

Las dos columnas preparan las mismas imágenes y conservan la partición del caso.

Programa · definir la red y aprender

Python

```
red = MLPRegressor(
    hidden_layer_sizes=capas,
    activation=ACTIVACION,
    solver="adam",
    learning_rate_init=TASA,
    batch_size=LOTE,
    alpha=ALPHA,
    random_state=SEMILLA,
    early_stopping=False,
)
```

R · RSNNS / stats

```
set.seed(SEMILLA)
red <- mlp(
  x = X_tr, y = X_tr,
  size = capas, maxit = epocas,
  hiddenActFunc = ACTIVACION,
  outputActFunc = "Act_IdentityPlusBias",
  learnFunc = "Std_Backpropagation",
  learnFuncParams = c(TASA, 0)
)
```

Python define el objeto; partial_fit lo entrenará. En R, mlp ya crea y entrena la candidata.

Programa · medir y seleccionar con validación

Python

```
mejor_mse = np.inf
for epoca in range(1, EPOCAS + 1):
    red.partial_fit(X_tr, X_tr)
    rec_va = red.predict(X_va)
    mse_va = mean_squared_error(
        X_va, rec_va)
    if mse_va < mejor_mse:
        mejor_mse = mse_va
        mejor_epoca = epoca
        red_elegida = deepcopy(red)
```

R · RSNNS / stats

```
rec_va <- predict(red, X_va)
mse_va <- mean((X_va - rec_va)^2)
if (mse_va < mejor_mse) {
  mejor_mse <- mse_va
  modelo <- red
  capas_elegidas <- capas
  epocas_elegidas <- epocas
}
```

Validación compara; entrenamiento aprende. R compara presupuestos de 25, 75 y 150 épocas.

Programa · reconstruir y comparar

Python

```
reconstruidas = modelo.predict(X_va)
mse_ae = mean_squared_error(
    X_va, reconstruidas)

pca = PCA(n_components=DIM_LATENTE,
          svd_solver="full")
pca.fit(X_tr)
Z_va = pca.transform(X_va)
rec_pca = pca.inverse_transform(Z_va)
```

R · RSNNS / stats

```
reconstruidas <- predict(modelo, X_va)
mse_ae <- mean((X_va - reconstruidas)^2)

pca <- prcomp(X_tr, center = TRUE,
  scale. = FALSE, rank. = DIM_LATENTE)
Z_va <- predict(pca, X_va)
rec_pca <- sweep(Z_va %*% t(pca$rotation),
  2, pca$center, "+")
```

Ambas redes reconstruyen. PCA ofrece la misma referencia lineal en Python y en R.

Práctica · cambiar el tamaño del resumen

Python

```
DIM_LATENTE = 4
ARQUITECTURAS = [
    (DIM_LATENTE,),
    (32, DIM_LATENTE, 32)
]
EXPERIMENTO = "dimension_4"
```

R · RSNNS / stats

```
DIM_LATENTE <- 4L
ARQUITECTURAS <- list(
  c(DIM_LATENTE),
  c(32L, DIM_LATENTE, 32L)
)
EXPERIMENTO <- "dimension_4"
```

Repetir 02–06. Comparar el error de validación y los detalles que conserva la imagen.